First Year Master of Technology
Lab Assignment 2

Course Code: 2304514L
Course Name: Machine Learning Laboratory
Academic Year: 2025–26

Student Details

Student Name: Utkarsh Bele

PRN Number: 202503040003

Title: Classification Models on Heart Disease dataset

Objective

To implement and analyze various classification algorithms including Support Vector Machine (SVM), Decision Tree, k-Nearest Neighbors (KNN), and Naïve Bayes on a real-world dataset, and to evaluate their performance using appropriate metrics such as accuracy, precision, recall, and F1-score in order to compare their effectiveness and suitability for different types of classification problems.

Problem Statement

Implement SVM, Decision Trees, k-Nearest Neighbors (KNN), and Naive Bayes classifiers on a real-world dataset and evaluate their performance using relevant metrics. The goal is to compare these models and discuss their suitability for different classification tasks.

Code

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# Models
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

# Load Dataset
df = pd.read_csv('/content/heart.csv')
df.head()

# Features & Target
X = df.drop('target', axis=1)
```

```
y = df['target']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Models
svm = SVC()
svm.fit(X_train, y_train)
svm_pred = svm.predict(X_test)

dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)

nb = GaussianNB()
nb.fit(X_train, y_train)
nb_pred = nb.predict(X_test)

# Evaluation Function
def evaluate(y_test, y_pred, model_name):
    print(f"\n{model_name} Performance:")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred))
    print("Recall:", recall_score(y_test, y_pred))
    print("F1 Score:", f1_score(y_test, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

evaluate(y_test, svm_pred, "SVM")
evaluate(y_test, dt_pred, "Decision Tree")
evaluate(y_test, knn_pred, "KNN")
evaluate(y_test, nb_pred, "Naive Bayes")
```
Output

```
SVM Performance:
Accuracy: 0.8878048780487805
Precision: 0.8508771929824561
Recall: 0.941747572815534
F1 Score: 0.8940092165898618
Confusion Matrix:
 [[85 17]
 [ 6 97]]

Decision Tree Performance:
Accuracy: 0.9853658536585366
Precision: 1.0
Recall: 0.970873786407767
F1 Score: 0.9852216748768473
Confusion Matrix:
 [[102   0]
 [  3 100]]

KNN Performance:
Accuracy: 0.8341463414634146
Precision: 0.8
Recall: 0.8932038834951457
F1 Score: 0.8440366972477065
Confusion Matrix:
 [[79 23]
 [11 92]]

Naive Bayes Performance:
Accuracy: 0.8
Precision: 0.7540983606557377
Recall: 0.8932038834951457
F1 Score: 0.8177777777777778
Confusion Matrix:
 [[72 30]
 [11 92]]
```

Model	Accuracy	Precision	Recall	F1 Score
SVM	0.888	0.851	0.942	0.894
Decision Tree	**0.985**	**1.000**	0.971	**0.985**
KNN	0.834	0.800	0.893	0.844
Naive Bayes	0.800	0.754	0.893	0.818

1. Best Performing Model

Decision Tree performed the best

- Highest **Accuracy (98.5%)**

- Perfect **Precision (1.0)** → No false positives
- Highest **F1 Score** Interpretation:

The model correctly classifies almost all cases and makes highly confident predictions.

2. SVM Performance

- High **Recall (0.94)** → detects most positive cases
- Slightly lower precision → some false positives

Meaning:

SVM is good when missing positive cases is risky (e.g., disease detection).

3. KNN Performance

- Moderate accuracy (83%)
- Balanced but weaker than SVM & DT

Reason:

- Sensitive to:
    - Choice of **k**
    - Feature scaling

4. Naïve Bayes Performance

- Lowest accuracy (80%)
- Lower precision

Reason:

Assumption of **feature independence is not valid** for this dataset.

Writes on following points

1. How does the choice of kernel affect the performance of an SVM classifier?

The **kernel function** determines how input data is transformed into a higher-dimensional feature space, enabling the SVM to find an optimal separating hyperplane.

- **Linear Kernel**
  Works well when data is linearly separable. Fast and less prone to overfitting.
- **Polynomial Kernel**
  Captures interactions between features. Useful for moderately complex relationships but can overfit if degree is high.

- **RBF (Radial Basis Function) Kernel**
  Most commonly used. Handles non-linear data effectively by creating flexible decision boundaries.
- **Sigmoid Kernel**
  Similar to neural networks but less commonly used in practice.

Impact:
Choosing the wrong kernel can lead to:

- Underfitting (too simple, e.g., linear for non-linear data)
- Overfitting (too complex, e.g., high-degree polynomial)

2. Why might Decision Trees overfit, and how can this be controlled?

Decision Trees overfit because they:

- Split data until **pure leaf nodes** are achieved
- Capture **noise and outliers** as if they were meaningful patterns

Control Techniques:

- **Max Depth** → Limit tree height
- **Min Samples Split / Leaf** → Prevent very small splits
- **Pruning** → Remove unnecessary branches
- **Use Ensemble Methods** → e.g., Random Forest

3. In KNN, how does the value of k influence the bias-variance tradeoff?

The value of **k (number of neighbors)** directly controls model complexity:

- **Small k (e.g., k=1)**
  - Low bias, high variance
  - Very sensitive to noise → overfitting
- **Large k**
  - High bias, low variance
  - Smoother decision boundary → underfitting

Trade-off:

- Small k → memorizes data

- Large k → generalizes but may ignore local patterns

4. What assumptions does Naïve Bayes make about the features, and when might these assumptions fail?

Naïve Bayes assumes:

Feature Independence
All features are conditionally independent given the class label.

Equal Contribution
Each feature contributes independently to the probability.

Conclusion

The practical successfully implemented multiple classification algorithms on the Heart Disease dataset. Decision Tree achieved the highest accuracy and precision, making it the best-performing model for this dataset. However, it may suffer from overfitting due to its high complexity. SVM demonstrated strong recall, making it suitable for applications where detecting positive cases is critical. KNN and Naïve Bayes showed comparatively lower performance due to sensitivity to parameters and underlying assumptions. Overall, model selection should be based on both performance metrics and the nature of the problem.